

# Assignment 1

STAT 8670

Tommy Stell  
Georgia State University

09 September, 2025

## Contents

|          |             |          |
|----------|-------------|----------|
| <b>1</b> | <b>Q1</b>   | <b>2</b> |
| <b>2</b> | <b>Q2</b>   | <b>6</b> |
| 2.1      | a . . . . . | 6        |
| 2.2      | b . . . . . | 6        |
| 2.3      | c . . . . . | 7        |
| <b>3</b> | <b>Q3</b>   | <b>8</b> |

```
# Assignment 1 for STAT 8670 - due 9/12/2025  
# all work by Tommy Stell
```

```
# loading packages/libraries  
options(repos = c(CRAN = "https://cloud.r-project.org/"))  
  
install.packages("pandoc")
```

```
## Installing package into 'C:/Users/tomle/AppData/Local/R/win-library/4.5'  
## (as 'lib' is unspecified)
```

```
## package 'pandoc' successfully unpacked and MD5 sums checked  
##  
## The downloaded binary packages are in  
## C:\Users\tomle\AppData\Local\Temp\RtmpWs05iF\downloaded_packages
```

```
library(microbenchmark)  
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0

## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
## explicit

library(ggplot2)

# loading the iris dataset
Q1_data <- iris
```

## 1 Q1

```
# 1.a) five summary statistics using base R
summary(Q1_data)
```

```
##   Sepal.Length   Sepal.Width   Petal.Length   Petal.Width
##   Min.      :4.300   Min.      :2.000   Min.      :1.000   Min.      :0.100
##   1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
##   Median :5.800   Median :3.000   Median :4.350   Median :1.300
##   Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
##   3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
##   Max.    :7.900   Max.    :4.400   Max.    :6.900   Max.    :2.500
##           Species
##   setosa      :50
##   versicolor :50
##   virginica   :50
##
##
##
```

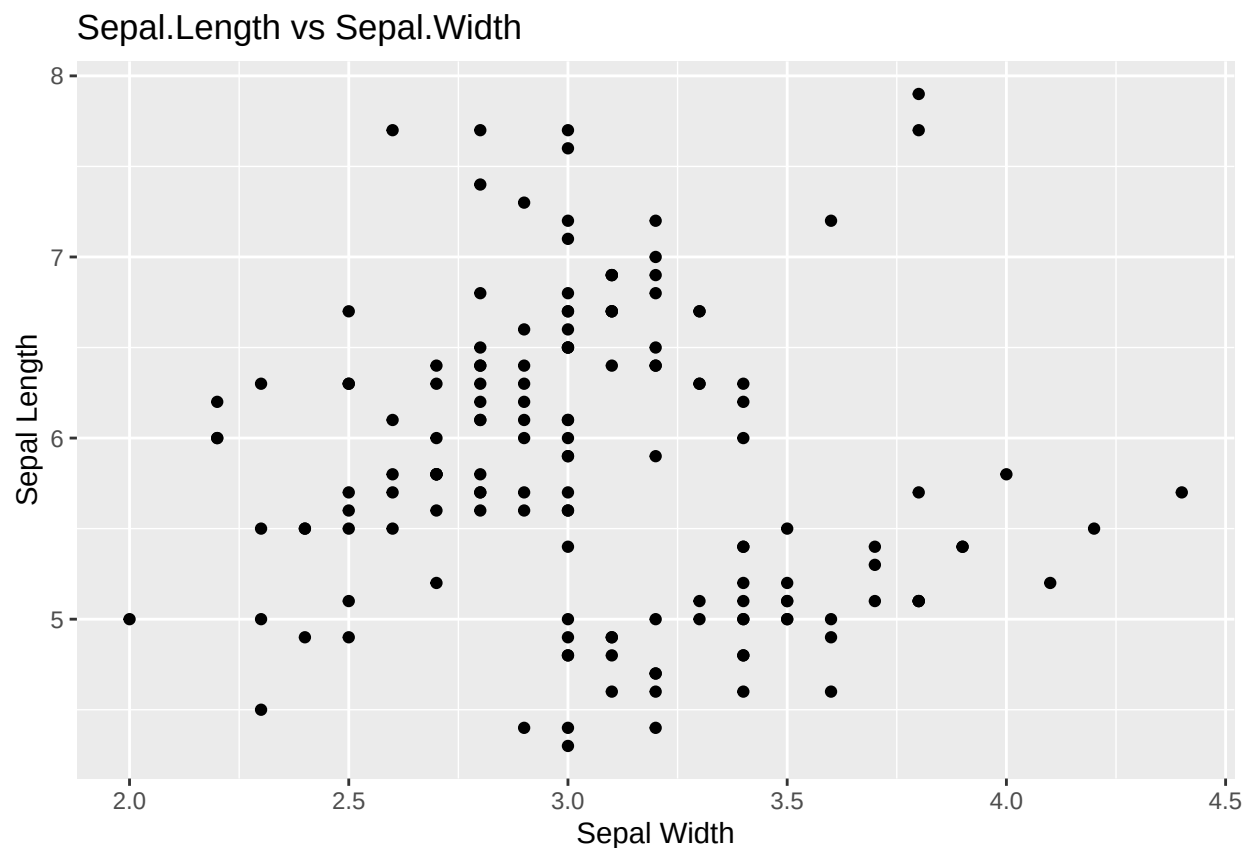
```
# 1.b) five summary statistics using tidyverse
Q1_data %>%
  summarise(
    Sepal.Length = list(summary(Sepal.Length)),
    Sepal.Width = list(summary(Sepal.Width)),
    Petal.Length = list(summary(Petal.Length)),
    Petal.Width = list(summary(Petal.Width)),
```

```
Species = list(summary(Species))
)
```

| Sepal.Length        | Sepal.Width         | Petal.Length  | Petal.Width         | Species |
|---------------------|---------------------|---------------|---------------------|---------|
| 4.300000, 5.100000, | 2.000000, 2.800000, | 1.000, 1.600, | 0.100000, 0.300000, | 50,     |
| 5.800000, 5.843333, | 3.000000, 3.057333, | 4.350, 3.758, | 1.300000, 1.199333, | 50,     |
| 6.400000, 7.900000  | 3.300000, 4.400000  | 5.100, 6.900  | 1.800000, 2.500000  | 50      |

```
# 1.c) use ggplot2 to create a scatter plot of Sepal.Length vs Sepal.Width
```

```
ggplot(Q1_data, aes(x = Sepal.Width, y = Sepal.Length)) +
  geom_point() +
  labs(title = "Sepal.Length vs Sepal.Width",
       x = "Sepal Width",
       y = "Sepal Length")
```



```
ggsave("Assignment_1_1c_scatter_plot.png")
```

```
## Saving 6.5 x 4.5 in image
```

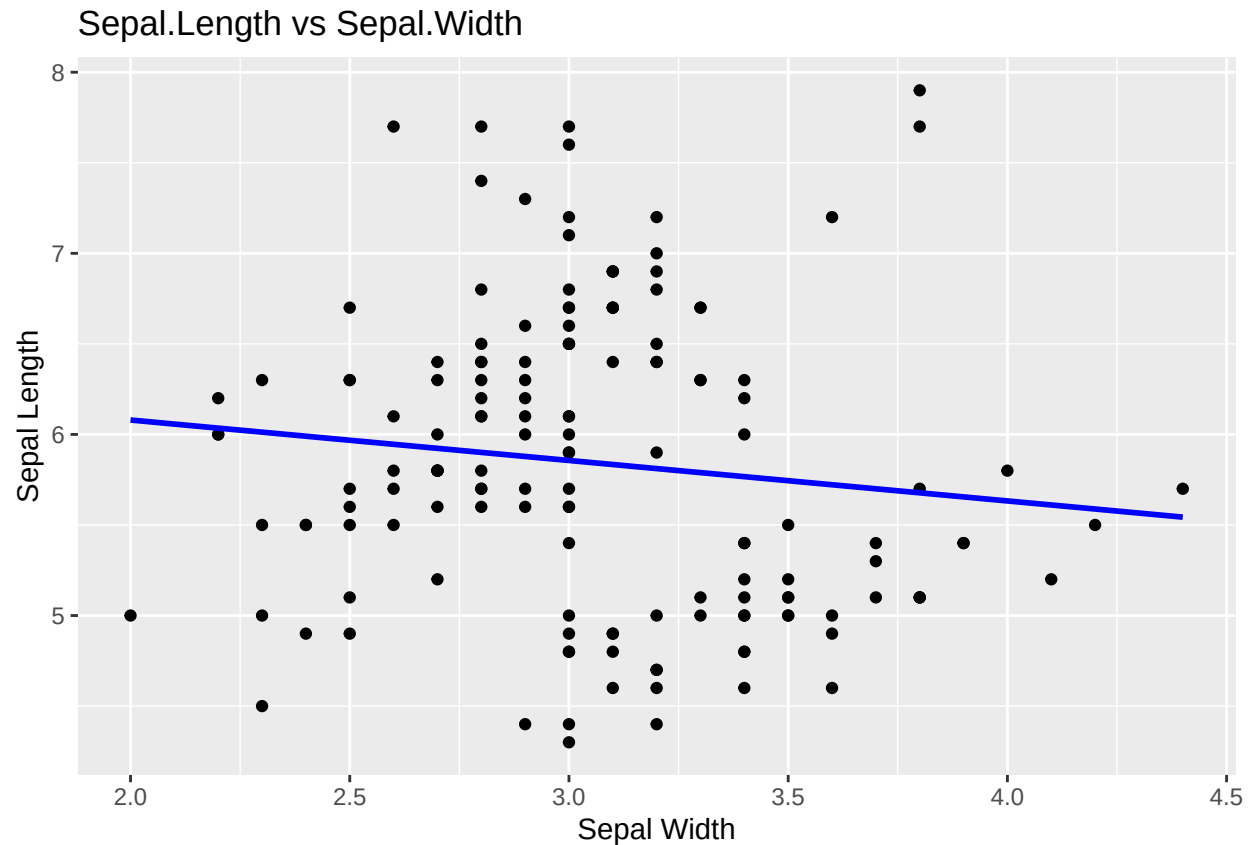
```
# 1.d) fit a linear regression with Length as response and Width as predictor
lm_model <- lm(Sepal.Length ~ Sepal.Width, data = Q1_data)
summary(lm_model)
```

```
##
## Call:
## lm(formula = Sepal.Length ~ Sepal.Width, data = Q1_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.5561 -0.6333 -0.1120  0.5579  2.2226
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   6.5262     0.4789   13.63  <2e-16 ***
## Sepal.Width  -0.2234     0.1551   -1.44    0.152
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8251 on 148 degrees of freedom
## Multiple R-squared:  0.01382,    Adjusted R-squared:  0.007159
## F-statistic: 2.074 on 1 and 148 DF,  p-value: 0.1519
```

```
# 1.e) add fitted regression line to the plot from part c
```

```
ggplot(Q1_data, aes(x = Sepal.Width, y = Sepal.Length)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  labs(title = "Sepal.Length vs Sepal.Width",
       x = "Sepal Width",
       y = "Sepal Length")
```

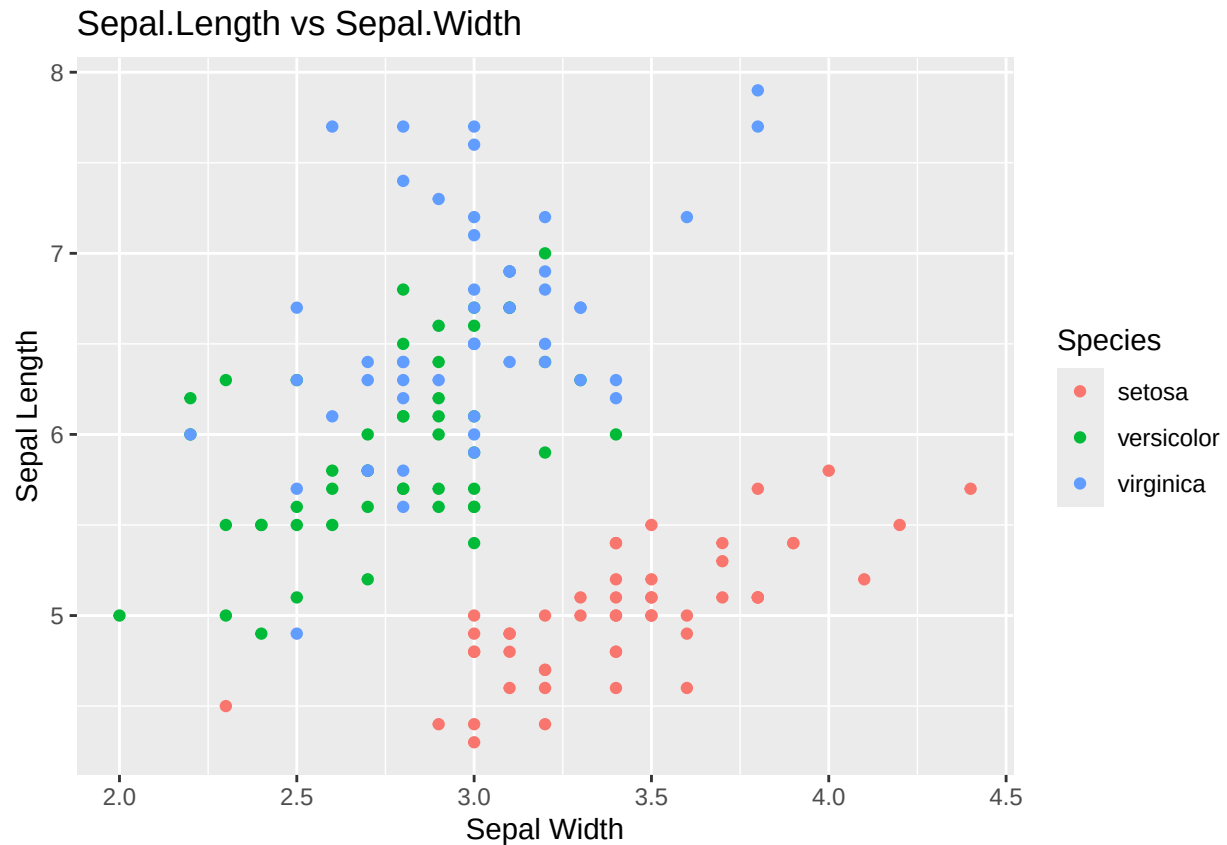
```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
ggsave("Assignment_1_1e_scatter_plot_with_regression_line.png")
```

```
## Saving 6.5 x 4.5 in image
## 'geom_smooth()' using formula = 'y ~ x'
```

```
# 1.f) use different colors to distinguish species in the plot
ggplot(Q1_data, aes(x = Sepal.Width, y = Sepal.Length, color = Species)) +
  geom_point() +
  #geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Sepal.Length vs Sepal.Width",
       x = "Sepal Width",
       y = "Sepal Length")
```



```
ggsave("Assignment_1_1f_scatter_plot_with_regression_line.png")
```

```
## Saving 6.5 x 4.5 in image
```

## 2 Q2

### 2.1 a

My research is in machine learning and the usage of statistical computing allows for me to actually be able to computationally approach problems. An example that I used to explain it to my wife is the usage of generative AI for generating images uses the computational statistics to function.

### 2.2 b

The first reason that could cause significant differences between computational and theoretical results is numerical imprecision (i.e. floating point error propagation).

The second reason is data imperfections since models are only as good as the data that is fed into them. If there are data quality issues, then no amount of perfect computational power could ever surpass the poor data.

The third reason is that the model could be perfectly accurate in an idealized setting but the real world application of the model will fail due to a violation of the assumptions leading to the idealized setting.

## 2.3 c

```
# Problem 2 - conceptual

# 2.c)

set.seed(777)
# I personally always use 137 as my set seed for
# anything like this since its tattooed on my arm

X <- matrix(rnorm(1000*100), 1000, 100)
y <- rnorm(1000)

normal_eqn <- function(X, y) {
  solve(t(X) %*% X) %*% t(X) %*% y
}

g_elim <- function(X,y){
  XtX <- t(X) %*% X
  Xty <- t(X) %*% y
  solve(XtX, Xty)
}

qr_decomp <- function(X,y){
  qr_X <- qr(X)
  qr.coef(qr_X, y)
}

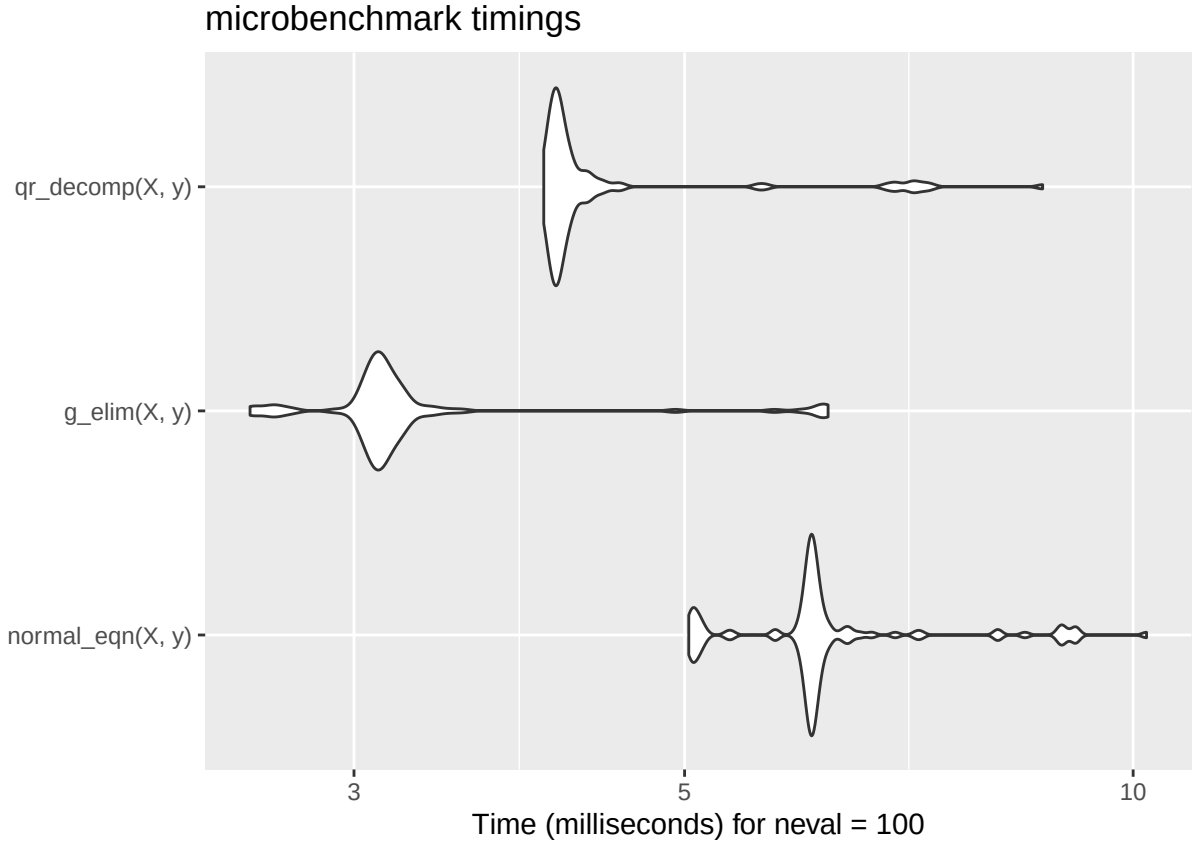
benchmark_results <- microbenchmark(
  normal_eqn(X, y),
  g_elim(X, y),
  qr_decomp(X, y),
  times = 100
)

print(benchmark_results)
```

```
## Unit: milliseconds
##          expr      min       lq      mean  median       uq      max neval
## normal_eqn(X, y) 5.0325 6.01845 6.284749 6.09200 6.17940 10.2094   100
##      g_elim(X, y) 2.5549 3.07100 3.325805 3.12275 3.22045  6.2413   100
##   qr_decomp(X, y) 4.0222 4.08175 4.454858 4.11955 4.22875  8.6950   100
```

```
autoplot(benchmark_results)
```



### 3 Q3

Given that  $\Sigma$  is a variance-covariance matrix, it is square i.e.  $\Sigma \in \mathcal{R}^{n \times n}$

By Cholesky Decomposition,  $\Sigma = R^T R$ , where  $R$  is upper triangular. This means the following hold.

$$\det(\Sigma) = \det(R^T R) = \det(R^T) \det(R) = \det(R)^2$$

Which when the log is taken of  $\det(\Sigma)$ , we get the following.

$$\ln(\det(\Sigma)) = 2\ln(\det(R))$$

Since  $R$  is a triangular matrix, the determinant of it is simply given as the product of the triangular entries:  $\det(R) = \prod_{i=1}^n r_{ii}$

This means that the  $\ln(\det(R)) = \sum_{i=1}^n \ln(r_{ii})$

Which finally means that the log-determinant of the variance-covariance matrix is two times the log of the sum of the diagonal entries of  $R$ .



$$\ln(\det(\Sigma)) = 2 \sum_{i=1}^n \ln(r_{ii})$$